IBOOKTRACK · HANDOVER DOCUMENTATION

Migration & Turnover Guide

A unified Warehouse Management System and Executive Dashboard for Innodata's book-digitization pipeline.

Generated 2026-05-15 · John Rey Cutab & Manwarul Francis Santos

iBookTrack — Migration & Turnover Guide

doc migration audience DevOps difficulty medium

Step-by-step playbook for moving iBookTrack to a new environment (new Supabase project, new server, new domain) **without losing data**.

Migration at a glance



0. Pre-Flight

Before you start, gather these from the current owner:

- [ ] Access to the **current** Supabase project (or a recent backup).
- [ ] The current `.env` file (or values for every variable in `PROJECT_OVERVIEW.md` §5).
- [ ] Repo access (Git).
- [ ] Domain DNS access if you're keeping the same URL.
- [ ] Innodata Simflex API key + config code (currently hard-coded — see cleanup notes).
- [ ] iLMS credentials if applicable.

1. Provision the New Database (Supabase)

1a. Create a fresh Supabase project

1. https://supabase.com → New Project.
2. Pick a strong DB password and **save it in a vault**.
3. From **Project Settings** → **API** copy:
   - `Project URL` → `NEXT_PUBLIC_SUPABASE_URL`
   - `anon public` key → `NEXT_PUBLIC_SUPABASE_ANON_KEY`
   - `service_role secret` key → `SUPABASE_SERVICE_ROLE_KEY`

1b. Run migrations on the new DB

In **SQL Editor** of the new project, run **every** file in `backend/migrations/` in numeric order (001 → 025). The numbering skips 020 — that gap is intentional.

You can also batch them via the Supabase CLI:

```
npm install -g supabase
supabase login
supabase link --project-ref <new-project-ref>
# Drop the SQL files into supabase/migrations/ and run:
supabase db push
```

After the run, verify the constraint that has bitten us repeatedly:

```
SELECT pg_get_constraintdef(oid)
FROM pg_constraint
WHERE conrelid = 'boxes'::regclass AND conname = 'boxes_current_stage_check';
```

It should list: `warehouse, inventory, cut, endorsement, scan, transmit, storage, intake`.

1c. Copy data from the old DB

Option A — Supabase native backup/restore (same provider, easiest):

1. Old project → Database → Backups → download a `.sql` dump.
2. New project → SQL Editor → paste & run, **after** the schema migrations.

Option B — `pg_dump` / `pg_restore` (cross-provider):

```
# Dump from source
pg_dump --host db.<old>.supabase.co --port 6543 --username postgres \
        --schema=public --data-only --no-owner --no-privileges \
        --format=custom --file=ibooktrack.dump postgres

# Restore into target
pg_restore --host db.<new>.supabase.co --port 6543 --username postgres \
           --dbname=postgres --no-owner --no-privileges --data-only \
           ibooktrack.dump
```

> **CAUTION**
>
> Don't dump `auth.*` schemas — Supabase manages those internally. Migrate users via step 1d instead.

Option C — selective table copy via SQL (when you only want recent rows):

```
-- Run on the OLD project
COPY (SELECT * FROM lots) TO STDOUT WITH CSV HEADER;
-- … then \COPY into the new DB.
```

1d. Migrating Supabase Auth users

Supabase Auth users live in `auth.users`. Use the dashboard's **Authentication** → **Users** → **Export** (CSV) and import into the new project, or use the `supabase-cli` migration script.

After migration, run on the new DB:

```
-- Backfill profiles from the imported auth.users (trigger 003 may already do this)
INSERT INTO profiles (id, email, name, role)
SELECT u.id, u.email, COALESCE(u.raw_user_meta_data->>'full_name', u.email), 'user'
FROM auth.users u
ON CONFLICT (id) DO NOTHING;
```

Promote admins:

```
UPDATE profiles SET role = 'admin' WHERE email IN (
  'admin1@innodata.com',
  'admin2@innodata.com'
);
```

---

2. Deploy the Backend

The backend is a plain Node.js Express app — pick any host that runs Node ≥ 20.

2a. Render / Railway / Fly.io

1. New service → connect the repo.
2. **Root directory:** `backend/`
3. **Build command:** `npm install`
4. **Start command:** `npm start`
5. **Environment variables** — copy from your new `.env`:

   ```
   NEXT_PUBLIC_SUPABASE_URL=...
   NEXT_PUBLIC_SUPABASE_ANON_KEY=...
   SUPABASE_SERVICE_ROLE_KEY=...
   PORT=5000
   FRONTEND_URL=https://app.example.com         # the deployed frontend
   FRONTEND_URLS=https://app.example.com,https://staging.example.com
   ```

6. Deploy. Hit `https://api.example.com/api/health` → expect `{ status: 'ok', timestamp: '…' }`.

2b. Self-hosted VPS

```
ssh root@server
git clone <repo> /opt/ibooktrack
cd /opt/ibooktrack/backend
npm ci --production
cp /opt/ibooktrack/.env.example /opt/ibooktrack/.env   # then edit

# Install pm2 (or systemd)
npm i -g pm2
pm2 start server.js --name ibooktrack-api
pm2 save && pm2 startup
```

Front it with **nginx** + a Let's Encrypt cert. Sample server block:

```
server {
  server_name api.example.com;
  listen 443 ssl;
  ssl_certificate     /etc/letsencrypt/live/api.example.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/api.example.com/privkey.pem;

  location / {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $remote_addr;
  }
}
```

---

3. Deploy the Frontend

3a. Vercel (recommended)

1. Import the repo. **Root directory:** `frontend/`.
2. Environment variables:

   ```
   NEXT_PUBLIC_SUPABASE_URL=...
   NEXT_PUBLIC_SUPABASE_ANON_KEY=...
   NEXT_PUBLIC_API_URL=https://api.example.com
   ```

3. Build command: auto (`next build`).
4. Add the production domain.

> ⚠ **WARNING**
>
> **Casing fix before Linux deploy:** rename `frontend/app/Dashboard/` → `frontend/app/dashboard/` and update all 25 references (search the repo for `'/Dashboard'`). Vercel runs Linux; case-sensitivity **will** break routes on first deploy.

3b. Self-hosted Node

```
cd /opt/ibooktrack/frontend
npm ci
npm run build
pm2 start "npm start" --name ibooktrack-web
```

Reverse-proxy as above (port 3000).

---

4. Pre-Flight Checks (Smoke Tests)

After both services are live:

Check	Expect
`curl https://api.example.com/api/health`	`{"status":"ok",…}`
`https://app.example.com/landing-page` loads	Marketing page
Register new account	Lands on `/Dashboard`
Login → token saved in `localStorage`	DevTools confirms
Dashboard loads stats + transmission + flagged tables	All three populate
`/warehouse-inventory` → start session → scan ISBN	Box auto-created in DB
Flag scan → reopens with proper label (not raw key)	UI uses `FLAG_LABELS`
`/checkpoints` → advance a box	`pipeline_events` row inserted
Sidebar log out	Tokens cleared, redirect to /login

---

5. Cleanup Before Turnover

Walk through these items with the receiving team. They are reproduced from `PROJECT_OVERVIEW.md` §6; tick them as you go.

🔴 Security

- [ ] Move `SIMFLEX_API_KEY`, `SIMFLEX_CONFIG`, `SIMFLEX_API_URL` from `backend/services/simflex-db.js` into `.env`.
- [ ] Delete or sanitize `SCANNING_INTEGRATION.md` (contains stale credentials).
- [ ] Lock CORS down to explicit `FRONTEND_URL(S)`; remove the `isDevBrowserOriginAllowed` shortcut from `server.js` for prod.
- [ ] Rotate the Supabase **service role key** if the old one was shared with contractors.

🟡 Dead code

- [ ] `backend/test-warehouse.js` — delete or move to `backend/scripts/`.
- [ ] `backend/routes/index.js` — unused; delete or sync with reality.
- [ ] `frontend/components/dashboard-sidebar.tsx` `sampleNotifications` block — delete (lines ~54-100).

🟡 Casing / cross-platform

- [ ] Rename `frontend/app/Dashboard/` → `frontend/app/dashboard/`.
- [ ] Search & replace `/Dashboard` → `/dashboard` (25 occurrences).

🟡 Documentation drift

- [ ] Replace `[Your Database Choice]` in root `README.md` with "Supabase / Postgres".
- [ ] Update `backend/migrations/README.md` pipeline list to match constraint.

🟢 Schema hygiene

- [ ] Confirm `boxes_current_stage_check` allowed list matches §1b.
- [ ] Confirm `warehouse_book_scans.rejection_reason` legacy values are cleaned.
- [ ] Run `pg_dump` once per week (cron / GitHub Action) and store off-Supabase.

---

6. Rollback Plan

If the new deployment misbehaves:

1. **DNS:** revert `app.example.com` and `api.example.com` to the old IPs.
2. **Frontend:** Vercel keeps every deployment — promote the previous one.
3. **Database:** The old Supabase project should be left **read-only** for at least 7 days post-cutover. To go back, point `NEXT_PUBLIC_SUPABASE_URL` at it in both frontend and backend env vars and redeploy.

---

7. Day-2 Operations

Task	Where
Add a new pipeline stage	Update `STAGE_ORDER` in `backend/routes/boxes.js`, the constraint, and any UI lists.
Add a new flag reason	Append to `FLAG_ISSUES` in `frontend/app/warehouse-inventory/page.tsx` **and** `FLAG_LABELS` in `frontend/components/missing-item-log.tsx`.
Promote a user	`UPDATE profiles SET role='admin' WHERE email='…';`
Backfill missing book titles	`POST /api/warehouse/backfill-titles`
Investigate stuck box	`SELECT * FROM pipeline_events WHERE box_id = '…' ORDER BY created_at;`
Reset a stuck stage	`POST /api/boxes/:id/rollback` body `{ to_stage: '…' }`

8. Contact / Handoff

Original team (OJT, 2025-2026):

- **John Rey Cutab**
- **Manwarul Francis Santos**

Built with Windsurf AI pair-programming. The conversation history with each AI session is preserved if you need to re-read design decisions.

Welcome aboard, and good luck. 🚀